

V2.0 documentation

2.0

Generated by Doxygen 1.14.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 Student Class Reference	7
4.1.1 Member Function Documentation	8
4.1.1.1 iverstiRanka()	8
4.1.1.2 skaiciuokGalutinis()	8
4.2 Zmogus Class Reference	8
5 File Documentation	11
5.1 Header.h	11
5.2 menuGen.h	12
5.3 sort.h	13
5.4 Studentas.h	15
5.5 test.h	16
Index	19

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	8
Student	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Student		7
Zmogus		8

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

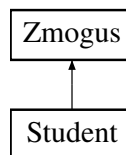
Header.h	11
menuGen.h	12
sort.h	13
Studentas.h	15
test.h	16

Chapter 4

Class Documentation

4.1 Student Class Reference

Inheritance diagram for Student:



Public Member Functions

- **Student** (const string &v, const string &p, const vector< int > &paz, int egz)
- **Student** (const [Student](#) &other)
- [Student](#) & **operator=** (const [Student](#) &other)
- **Student** ([Student](#) &&other) noexcept
- [Student](#) & **operator=** ([Student](#) &&other) noexcept
- double [skaiciuokGalutinis](#) (bool naudotiVidurki) override
- void [ivestiRanka](#) () override
- double **getGalutinisVid** () const
- double **getGalutinisMed** () const
- int **getEgzaminas** () const
- const vector< int > & **getPazymiai** () const
- void **setEgzaminas** (int e)
- void **addNd** (int nd)

Public Member Functions inherited from [Zmogus](#)

- **Zmogus** (const string &v, const string &p)
- **Zmogus** (const [Zmogus](#) &other)=default
- [Zmogus](#) & **operator=** (const [Zmogus](#) &other)=default
- **Zmogus** ([Zmogus](#) &&other) noexcept=default
- [Zmogus](#) & **operator=** ([Zmogus](#) &&other) noexcept=default
- void **setVardas** (const string &v)
- void **setPavarde** (const string &p)
- string **getVardas** () const
- string **getPavarde** () const

Static Public Member Functions

- static bool **nuskaitytilsFailo** (const string &filename, vector< [Student](#) > &grupe, bool naudotiVidurki)
- static void **Isvedimas** (const vector< [Student](#) > &grupe)

Friends

- std::ostream & **operator<<** (std::ostream &out, const [Student](#) &s)
- std::istream & **operator>>** (std::istream &in, [Student](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string **vardas**
- string **pavarde**

4.1.1 Member Function Documentation

4.1.1.1 ivestiRanka()

```
void Student::ivestiRanka () [override], [virtual]
```

Implements [Zmogus](#).

4.1.1.2 skaiciuokGalutinis()

```
double Student::skaiciuokGalutinis (
    bool naudotiVidurki) [override], [virtual]
```

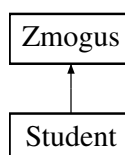
Implements [Zmogus](#).

The documentation for this class was generated from the following files:

- Studentas.h
- Studentas.cpp

4.2 Zmogus Class Reference

Inheritance diagram for Zmogus:



Public Member Functions

- **Zmogus** (const string &v, const string &p)
- **Zmogus** (const [Zmogus](#) &other)=default
- **Zmogus & operator=** (const [Zmogus](#) &other)=default
- **Zmogus** ([Zmogus](#) &&other) noexcept=default
- **Zmogus & operator=** ([Zmogus](#) &&other) noexcept=default
- virtual double **skaiciuokGalutinis** (bool naudotiVidurki)=0
- void **setVardas** (const string &v)
- void **setPavarde** (const string &p)
- string **getVardas** () const
- string **getPavarde** () const
- virtual void **ivestiRanka** ()=0

Protected Attributes

- string **vardas**
- string **pavarde**

The documentation for this class was generated from the following file:

- Studentas.h

Chapter 5

File Documentation

5.1 Header.h

```
00001 #pragma once
00002 #include <iostream>
00003 #include <vector>
00004 #include <string>
00005 #include <iomanip>
00006 #include <numeric>
00007 #include <algorithm>
00008 #include <random>
00009 #include <cstdlib>
00010 #include <ctime>
00011 //nuo v0.2
00012 #include <fstream>
00013 #include <limits>
00014 #include <sstream>
00015 #include <chrono> // testavimui
00016 #include <stdexcept> //
00017 #include <filesystem>
00018
00019 // #include <istream>
00020 #include "Studentas.h"
00021 #include <cassert> // for rule of five test
00022
00023 using std::cout;
00024 using std::string;
00025 using std::vector;
00026 using std::endl;
00027 using std::cin;
00028 using std::fixed;
00029 using std::setprecision;
00030 using std::left;
00031 using std::setw;
00032 using std::ifstream;
00033 using std::accumulate; // suma / .size -> vidurkis
00034 using std::ofstream;
00035 using std::cerr; // klaidoms
00036 using std::move;
00037
00038
00039 //std::uniform_int_distribution<> dis(0, 10); // sugeneruoja random skaičius nuo 0 iki 10 - pazymiams
00040 //std::uniform_int_distribution<> diss(1, 10); // - studentu kiekiui
00041
00042 using std::uniform_int_distribution;
00043 using std::random_device;
00044 using std::mt19937;
00045
00046 using std::chrono::high_resolution_clock;
00047 using std::chrono::duration_cast;
00048 using std::chrono::duration;
00049 using std::chrono::milliseconds;
00050
00051
00052 inline int m = 0; //studentu kiekis
00053 inline int nd = 0; //namu darbu rezultatai
00054 inline int pasirinkimas = 0; // skaiciuoti pagal vidurki ar mediana
00055 inline int isv = 0;
00056
00057 inline vector<Student> grupe;
```

```

00058 inline string choice;
00059 inline vector<int> paz_temp;
00060 inline string sortChoice;
00061 inline Student laik;
00062
00063
00064 // vardu ir pavardziu sarasas
00065 inline const vector<string> firstNames = { "Augustas", "Birute", "Daiva", "Ema", "Fiodoras",
    "Gabrielius", "Haroldas", "Ieva", "Tomas", "Niija" };
00066 inline const vector<string> lastNames = { "Baravykas", "Kiskis", "Lydeka", "Burokas", "Neris",
    "Jankauskas", "Kazlauskas", "Urbonas", "Boruta", "Zemaite" };
00067
00068 inline int testnr = 1;
00069

```

5.2 meniuGen.h

```

00001 #pragma once
00002 #include "Header.h"
00003 #include "Studentas.h"
00004
00005 // Meniu
00006 void meniu()
00007 {
00008     cout << "\n\nMeniu: \n";
00009     cout << "1 - Rankiniu budu ivesti pazymius\n";
00010     cout << "2 - Generuoti pazymius\n";
00011     cout << "3 - Generuoti pazymius ir studentu vardus bei pavardes\n";
00012     cout << "4 - Duomenis nuskaityti is failo\n";
00013     cout << "5 - BAIGTI darba ir isvesti rezultatus\n";
00014     cout << "6 - Sugeneruoti testavimo failus\n";
00015     cout << "7 - Isvalyti ivestus duomenis\n";
00016     cout << "8 - 'Rule of five' ir ivesties/ivesties operatoriu veikimo testavimas\n";
00017
00018     cout << "Pasirinkite (1, 2, 3, 4, 5, 6, 7, 8): ";
00019     cin >> choice;
00020 }
00021
00022 // Pasirenkamas nuskaitymo failas
00023 string pasirinktiFaila() {
00024     int nus = 0;
00025     string filename;
00026
00027     while (true) {
00028         cout << "\nPasirinkite koki faila nuskaitysite:\n"
00029             << "1 - kursiokai.txt\n"
00030             << "2 - studentail0000.txt\n"
00031             << "3 - studentail00000.txt\n"
00032             << "4 - studentail000000.txt\n"
00033             << "5 - pats ivesiu pavadinima.\n";
00034         cin >> nus;
00035
00036         if (nus == 1) return "kursiokai.txt";
00037         if (nus == 2) return "studentail0000.txt";
00038         if (nus == 3) return "studentail00000.txt";
00039         if (nus == 4) return "studentail000000.txt";
00040         if (nus == 5) {
00041             cin >> filename;
00042             return filename;
00043         }
00044         cout << "Neteisingas pasirinkimas. Bandykite dar karta.\n";
00045     }
00046 }
00047
00048 // Atsitiktiniam skaiciavimui
00049 random_device rd;
00050 mt19937 gen(rd());
00051 uniform_int_distribution<> diss(1, 10); // sugeneruoja random skaicius nuo 0 iki 10 - pazymiams
00052 uniform_int_distribution<> dis(0, 10); // - studentu kiekiui
00053
00054 bool naudotiVidurki = true;
00055 // Tik vidurkis ar mediana pasirenkama
00056 void vidarmed(bool& naudotiVidurki) {
00057     while (true) {
00058         cout << "\nPasirinkite skaiciavimo buda (1 - Vidurkis, 2 - Mediana): ";
00059         cin >> pasirinkimas;
00060         if (pasirinkimas == 1) {
00061             naudotiVidurki = true;
00062             break;
00063         }
00064         else if (pasirinkimas == 2) {
00065             naudotiVidurki = false;
00066             break;
00067         }
00068     }
00069 }

```



```

00067     }
00068     else {
00069         cout << "Netinkama verte. Iveskite skaiciu 1 arba 2.\n";
00070     }
00071 }
00072 }
00073
00074 // Generuoja pazymius Student objektui
00075 void pazymiai(Student& a) {
00076     int ndKiek = diss(gen); // kiekis
00077     cout << "- Namu darbu rezultatai (atsitiktiniai, 0-10): ";
00078
00079     // random namu darbu rezultatai
00080     for (int i = 0; i < ndKiek; ++i) {
00081         int paz = dis(gen);
00082         a.addNd(paz);
00083         cout << paz << " ";
00084     }
00085     // random egzamino pazymys
00086     int egz = dis(gen);
00087     a.setEgzaminas(egz);
00088     cout << "\n- Egzamino rezultatas (atsitiktinis, 0-10): " << egz << endl;
00089 }
00090
00091
00092 // Duomeniu failu generavimas
00093 void generateStudentFile(const std::string& filename, size_t recordCount) {
00094     std::ofstream out(filename);
00095     if (!out.is_open()) {
00096         std::cerr << "Nepavyko atidaryti failo: " << filename << "\n";
00097         return;
00098     }
00099
00100     std::random_device rd;
00101     std::mt19937 gen(rd());
00102     std::uniform_int_distribution<> dis(1, 10);
00103
00104     for (size_t i = 1; i <= recordCount; ++i) {
00105         out << "VardasNR" << i << " PavardeNR" << i << " "
00106             << dis(gen) << " " << dis(gen) << " "
00107             << dis(gen) << " " << dis(gen) << " "
00108             << dis(gen) << "\n";
00109     }
00110
00111     out.close();
00112     cout << "\nFailas sukurtas: " << filename << " (" << recordCount << " studentu)";
00113 }
00114 inline void GenFailai() {
00115     std::vector<std::pair<std::string, size_t>> failai = {
00116         {"studentai_1k.txt", 1'000},
00117         {"studentai_10k.txt", 10'000},
00118         {"studentai_100k.txt", 100'000},
00119         {"studentai_1m.txt", 1'000'000},
00120         {"studentai_10m.txt", 10'000'000}
00121     };
00122
00123     for (const auto& [filename, count] : failai) {
00124         auto start = std::chrono::high_resolution_clock::now();
00125         generateStudentFile(filename, count);
00126         auto end = std::chrono::high_resolution_clock::now();
00127         std::chrono::duration<double> elapsed = end - start;
00128         cout << "Sugeneruota per: " << elapsed.count() << " s\n\n";
00129     }
00130 }
00131

```

5.3 sort.h

```

00001 #pragma once
00002 #include "Header.h"
00003 #include "Studentas.h"
00004
00005 // Rikiavimo pasirinkimas
00006 void SortMenu(char& sortChoice) {
00007
00008     while (true) {
00009         cout << "\nPasirinkite pagal ka norite surusiuoti: \n a) Pavarde \n b) Varda \n c) Galutinis
00010 pagal vidurki \n d) Galutinis pagal mediana. \nIveskite tik pasirinkimo raide: ";
00011         cin >> sortChoice;
00012
00013         // tikrinama ar ivestis atitinka galimus paasirinkimus
00014         if (sortChoice == 'a' || sortChoice == 'b' || sortChoice == 'c' || sortChoice == 'd') {
00015             break;
00016         }
00017     }
00018 }

```

```

00015     }
00016     else {
00017         cout << "\nNeteisingas pasirinkimas. Bandykite dar karta.\n";
00018         cin.clear();
00019         cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // ignoruoja neteisinga
    investi
00020     }
00021 }
00022 }
00023
00024 // Isrenkami tik skaitmenys
00025 int extractNumber(const string& pavarde) {
00026     string digits;
00027     for (char c : pavarde) {
00028         if (isdigit(c)) digits += c;
00029     }
00030     return digits.empty() ? 0 : std::stoi(digits);
00031 }
00032
00033 // Rikiavimas
00034 void Sort(char sortChoice, vector<Student>& grupe) {
00035     if (sortChoice == 'a') { //pavarde
00036         sort(grupe.begin(), grupe.end(), [](const Student& a, const Student& b) {
00037             int numA = extractNumber(a.getPavarde());
00038             int numB = extractNumber(b.getPavarde());
00039             if (numA == numB) return a.getPavarde() < b.getPavarde(); // turi sutapti skaitmenu
    kiekis
00040             return numA < numB;
00041         });
00042     }
00043     else if (sortChoice == 'b') { //vardas
00044         sort(grupe.begin(), grupe.end(), [](const Student& a, const Student& b) {
00045             int numA = extractNumber(a.getVardas());
00046             int numB = extractNumber(b.getVardas());
00047             if (numA == numB) return a.getVardas() < b.getVardas(); // turi sutapti skaitmenu kiekis
00048             return numA < numB;
00049         });
00050     }
00051     else if (sortChoice == 'c') { // vidurkio galutinis
00052         sort(grupe.begin(), grupe.end(), [](const Student& a, const Student& b) {
00053             return a.getGalutinisVid() < b.getGalutinisVid();
00054         });
00055     }
00056     else if (sortChoice == 'd') { // medianos galutinis
00057         sort(grupe.begin(), grupe.end(), [](const Student& a, const Student& b) {
00058             return a.getGalutinisMed() < b.getGalutinisMed();
00059         });
00060     }
00061 }
00062
00063
00064 // Isskirstymas i kietiakius ir vargsiukus pagal bala
00065 void Skirstymas(const vector<Student>& grupe) {
00066
00067     vector<Student> kietiakiiai;
00068     vector<Student> vargsiukai;
00069
00070     for (const auto& stud : grupe) {
00071         if (stud.getGalutinisVid() >= 5.0 || stud.getGalutinisMed() >= 5)
00072             kietiakiiai.push_back(stud);
00073         else
00074             vargsiukai.push_back(stud);
00075     }
00076     std::ofstream outKiet("kietiakiiai.txt");
00077     std::ofstream outVargs("vargsiukai.txt");
00078
00079     if (!outKiet) {
00080         std::cout << "Klaida atidarant kietiakiiai.txt faila!\n";
00081         return;
00082     }
00083     if (!outVargs) {
00084         std::cout << "Klaida atidarant vargsiukai.txt faila!\n";
00085         return;
00086     }
00087
00088     // Kietiakiiai
00089     outKiet << left << setw(15) << "\nPavarde" << setw(15) << "Vardas";
00090     outKiet << setw(20) << "Galutinis (Vid.)" << setw(20) << "Galutinis (Med.)" << endl;
00091     outKiet << "-----\n";
00092
00093     for (const auto& s : kietiakiiai) {
00094         outKiet << left << setw(15) << s.getPavarde() << setw(15) << s.getVardas();
00095         // galutinis pasirinktas pagal vidurki, o prie medianos - x
00096         if (s.getGalutinisVid() == -1) outKiet << setw(20) << "x.xx";
00097         else outKiet << setw(20) << fixed << setprecision(2) << s.getGalutinisVid();
00098         // galutinis pasirinktas pagal mediana, o prie vidurkio - x
00099         if (s.getGalutinisMed() == -1) outKiet << setw(20) << "x.xx" << endl;
    }
}

```

```

00100         else outKiet << setw(20) << fixed << setprecision(2) << s.getGalutinisMed() << endl;
00101     }
00102
00103     // Vargsiukai
00104     outVargs << left << setw(15) << "\nPavarde" << setw(15) << "Vardas";
00105     outVargs << setw(20) << "Galutinis (Vid.)" << setw(20) << "Galutinis (Med.)" << endl;
00106     outVargs << "-----\n";
00107
00108     for (const auto& s : vargsiukai) {
00109         outVargs << left << setw(15) << s.getPavarde() << setw(15) << s.getVardas();
00110         // galutinis pasirinktas pagal vidurki, o prie medianos - x
00111         if (s.getGalutinisVid() == -1) outVargs << setw(20) << "x.xx";
00112         else outVargs << setw(20) << fixed << setprecision(2) << s.getGalutinisVid();
00113         // galutinis pasirinktas pagal mediana, o prie vidurkio - x
00114         if (s.getGalutinisMed() == -1) outVargs << setw(20) << "x.xx" << endl;
00115         else outVargs << setw(20) << fixed << setprecision(2) << s.getGalutinisMed() << endl;
00116     }
00117     //failas.close();
00118     cout << "\nPapildomai surusiuota ir isvesta i failus: \"kietiaki.txt\" (balas >= 5) ir
00119     \"vargsiukai.txt\" (balas < 5).\n";
00119 }

```

5.4 Studentas.h

```

00001 #pragma once
00002 #include <vector>
00003 #include <string>
00004 #include <iostream>
00005 #include <utility> // std::move
00006
00007 using std::string;
00008 using std::vector;
00009
00010 class Zmogus {
00011 protected:
00012     string vardas;
00013     string pavarde;
00014
00015 public:
00016     Zmogus() = default;
00017     Zmogus(const string& v, const string& p) : vardas(v), pavarde(p) {}
00018
00019     // RULE OF FIVE
00020     virtual ~Zmogus() = default; //destruktorius
00021     Zmogus(const Zmogus& other) = default; // Kopijavimo konstruktorius
00022     Zmogus& operator=(const Zmogus& other) = default; // Kopijavimo priskyrimo operatorius
00023     Zmogus(Zmogus&& other) noexcept = default; // Perkelimo konstruktorius
00024     Zmogus& operator=(Zmogus&& other) noexcept = default; // Perkelimo priskyrimo operatorius
00025
00026     // Abstraktus metodas, pavyzdziui, galutinis balas, kad klase butu abstrakti
00027     virtual double skaiciuokGalutinis(bool naudotiVidurki) = 0;
00028
00029     // Geteriai ir seteriai
00030     void setVardas(const string& v) { vardas = v; }
00031     void setPavarde(const string& p) { pavarde = p; }
00032
00033     string getVardas() const { return vardas; }
00034     string getPavarde() const { return pavarde; }
00035
00036     // Ivedimas ranka
00037     virtual void ivestiRanka() = 0;
00038 };
00039
00040 extern bool isTestavimoRezimas;
00041
00042 class Student : public Zmogus {
00043 private:
00044     vector<int> pazymiai;
00045     int egzaminas;
00046     double galutinisVid;
00047     double galutinisMed;
00048
00049 public:
00050     // Ivesties / Isvesties operatoriai
00051     friend std::ostream& operator<<(std::ostream& out, const Student& s); // isvesties
00052     friend std::istream& operator>>(std::istream& in, Student& s); // ivesties
00053
00054     // Konstruktoriai
00055     Student() = default; // default
00056     explicit Student(const string& v, const string& p, const vector<int>& paz, int egz); // pilnas
00057
00058     // Rule of five
00059 }

```

```

00060 ~Student() override; // = default; // destruktorius
00061 //~Student() = default;
00062 Student(const Student& other); // kopijavimo konstruktorius
00063 Student& operator=(const Student& other); // kopijavimo priskyrimo operatorius
00064 Student(Student&& other) noexcept; // perkeliimo konstruktorius
00065 Student& operator=(Student&& other) noexcept; // perkeliimo priskyrimo operatorius
00066
00067 // Implementuoti abstraktu metoda is Zmogus
00068 double skaiciuokGalutinis(bool naudotiVidurki) override;
00069
00070 // Ivesties metodai
00071 void ivestiRanka() override;
00072
00073 // Nuskaitymas is failo
00074 static bool nuskaitytiIsFailo(const string& filename, vector<Student>& grupe, bool
naudotiVidurki);
00075
00076 // Isvedimas
00077 static void Isvedimas(const vector<Student>& grupe);
00078
00079 // Get'ai
00080 double getGalutinisVid() const { return galutinisVid; }
00081 double getGalutinisMed() const { return galutinisMed; }
00082 int getEgzaminas() const { return egzaminas; }
00083 const vector<int>& getPazymiai() const { return pazymiai; }
00084
00085 // Nustatymai
00086 void setEgzaminas(int e) { egzaminas = e; }
00087 void addNd(int nd) { pazymiai.push_back(nd); }
00088 };

```

5.5 test.h

```

00001 #pragma once
00002 #include "Header.h"
00003 #include "Studentas.h"
00004
00005 // testavimui
00006 inline std::chrono::duration<double> diff; // skirtumas sekundemis
00007 inline vector<double> testai;
00008 double tvid(const vector<double>& times) {
00009     if (times.empty()) return 0.0;
00010     return accumulate(times.begin(), times.end(), 0.0) / times.size();
00011 }
00012
00013 /* testavimo rezultatai VECTOR:
00014 Vidutinis nuskaitymo vykdymo laikas 10 000 studentu per 5 testus: 3.14733 s
00015 Vidutinis nuskaitymo vykdymo laikas 100 000 studentu per 5 testus: 40.7781 s
00016 Vidutinis nuskaitymo vykdymo laikas 1 000 000 studentu per 5 testus: 206.372 s
00017 */
00018
00019 /* Nauju generuojamu duomeniu failu testavimo rezultatai:
00020 1 000 studentu: 0.0172659 s
00021 10 000 studentu: 0.145388 s
00022 100 000 studentu: 1.495 s
00023 1 000 000 studentu: 15.8693 s
00024 10 000 000 studentu: 165.408 s
00025 */
00026
00027 // RULE OF FIVE testavimui
00028 inline void testRuleOfFive() {
00029     cout << "\n--- Tikrinamas Rule of Five ---\n";
00030
00031     // Originalus objektas
00032     Student s1("Vardas", "Pavarde", { 10, 9, 8 }, 9);
00033     s1.skaiciuokGalutinis(true);
00034
00035     //Zmogus z; // Kompiliacijos klaida negalima sukurti abstrakcios klases objekto
00036
00037     // Kopijavimo konstruktorius
00038     Student s2(s1);
00039     if (s2.getVardas() != s1.getVardas())
00040         cerr << "Kopijavimo konstruktorius: vardas nesutampa\n";
00041     if (s2.getPavarde() != s1.getPavarde())
00042         cerr << "Kopijavimo konstruktorius: pavarde nesutampa\n";
00043     if (s2.getEgzaminas() != s1.getEgzaminas())
00044         cerr << "Kopijavimo konstruktorius: egzaminas nesutampa\n";
00045     if (s2.getPazymiai() != s1.getPazymiai())
00046         cerr << "Kopijavimo konstruktorius: pazymiai nesutampa\n";
00047     if (s2.getGalutinisVid() != s1.getGalutinisVid())
00048         cerr << "Kopijavimo konstruktorius: galutinisVid nesutampa\n";
00049     if (s2.getGalutinisMed() != s1.getGalutinisMed())
00050         cerr << "Kopijavimo konstruktorius: galutinisMed nesutampa\n";

```

```

00051
00052 // Kopijavimo priskyrimo operatorius
00053 Student s3;
00054 s3 = s1;
00055 if (s3.getVardas() != s1.getVardas())
00056     cerr << "Kopijavimo priskyrimo operatorius: vardas nesutampa\n";
00057 if (s3.getPavarde() != s1.getPavarde())
00058     cerr << "Kopijavimo priskyrimo operatorius: pavarde nesutampa\n";
00059 if (s3.getEgzaminas() != s1.getEgzaminas())
00060     cerr << "Kopijavimo priskyrimo operatorius: egzaminas nesutampa\n";
00061 if (s3.getPazymiai() != s1.getPazymiai())
00062     cerr << "Kopijavimo priskyrimo operatorius: pazymiai nesutampa\n";
00063 if (s3.getGalutinisVid() != s1.getGalutinisVid())
00064     cerr << "Kopijavimo priskyrimo operatorius: galutinisVid nesutampa\n";
00065 if (s3.getGalutinisMed() != s1.getGalutinisMed())
00066     cerr << "Kopijavimo priskyrimo operatorius: galutinisMed nesutampa\n";
00067
00068 // Perkelimo konstruktorius
00069 Student s4(std::move(s1));
00070 if (s4.getVardas() != "Vardas")
00071     cerr << "Perkelimo konstruktorius: vardas neteisingas po move\n";
00072 if (s4.getPavarde() != "Pavarde")
00073     cerr << "Perkelimo konstruktorius: pavarde nesutampa\n";
00074 if (s4.getEgzaminas() != 9)
00075     cerr << "Perkelimo konstruktorius: egzaminas nesutampa\n";
00076 if (s4.getPazymiai() != vector<int>({ 10, 9, 8 }))
00077     cerr << "Perkelimo konstruktorius: pazymiai nesutampa\n";
00078 if (s4.getGalutinisVid() <= 0.0)
00079     cerr << "Perkelimo konstruktorius: galutinisVid neapskaiciuotas\n";
00080
00081 // Perkelimo priskyrimo operatorius
00082 Student s5;
00083 s5 = std::move(s2);
00084 if (s5.getVardas() != "Vardas")
00085     cerr << "Perkelimo priskyrimo operatorius: vardas nesutampa\n";
00086 if (s5.getPavarde() != "Pavarde")
00087     cerr << "Perkelimo priskyrimo operatorius: pavarde neteisinga po move\n";
00088 if (s5.getEgzaminas() != 9)
00089     cerr << "Perkelimo priskyrimo operatorius: egzaminas nesutampa\n";
00090 if (s5.getPazymiai() != vector<int>({ 10, 9, 8 }))
00091     cerr << "Perkelimo priskyrimo operatorius: pazymiai nesutampa\n";
00092 if (s5.getGalutinisVid() <= 0.0)
00093     cerr << "Perkelimo priskyrimo operatorius: galutinisVid neapskaiciuotas\n";
00094
00095     cout << "\nRULE OF FIVE TESTAS SEKMINGAI BAIGTAS:\n";
00096 }
00097
00098 // Ivesties ir isvesties operatoriu testavimas
00099 inline void testOperatoriai() {
00100     cout << "\n--- Tikrinami ivesties ir isvesties operatoriai ---\n";
00101
00102     // Sukuriam teksto srauta su studento duomenimis (kaip ivestis)
00103     std::istringstream input("Vardas\nPavarde\n3\n8 9 10\n9\n");
00104
00105     Student s;
00106     // Ivedame duomenis i teksto srauto
00107     input >> s;
00108     if (input.fail()) {
00109         cerr << "\nKlaida: nepavyko nuskaityti studento duomenu.\n";
00110     }
00111
00112     // Patikriname ar teisingai nuskaityta
00113     assert(s.getVardas() == "Vardas");
00114     assert(s.getPavarde() == "Pavarde");
00115     assert(s.getEgzaminas() == 9);
00116     vector<int> tiketini = { 8, 9, 10 };
00117     assert(s.getPazymiai() == tiketini);
00118
00119     // Isvedame i tekstini srauta
00120     std::ostringstream output;
00121     output << s;
00122
00123     // Patikriname ar isvestis nera tuscia
00124     assert(!output.str().empty());
00125
00126     cout << "\n\nIVESTIES / ISVESTIES OPERATORIU TESTAS SEKMINGAS (su nustatytais duomenimis):\n\n" <<
00127         output.str() << endl;
00128 }

```


Index

ivestiRanka
Student, [8](#)

skaiciuokGalutinis
Student, [8](#)

Student, [7](#)
ivestiRanka, [8](#)
skaiciuokGalutinis, [8](#)

Zmogus, [8](#)